Ecommerce Platform - Developer Documentation

Ecommerce Platform — Developer Documentation
Table of Contents
README
API Reference
UML Diagrams
Usage Guide
Advanced / Reference

Ecommerce Platform — Developer Documentation

`Java 24` · `Spring Boot 3.4.0` · `React 19` · `Vite 8` · `MySQL 8.x` · `License: Unspecified`

A full-stack e-commerce platform consisting of a **Spring Boot 3 REST API backend** (`Ecommerce-backend`) and a **React 19 + Vite single-page application frontend** (`Ecommerce-frontend`), backed by MySQL and secured with JWT-based authentication.

> **A note on how this document was produced:** This documentation was generated from what could be verified directly from the two public repositories (`25karthik06/Ecommerce-backend` and `25karthik06/Ecommerce-frontend`) — primarily `pom.xml`, `package.json`, and root-level files. GitHub's automated-access protections prevented a full recursive crawl of `src/main/java` and `src/`, so the **API Reference** section below documents the *conventional REST shape* that this stack (Spring Boot + Spring Security + JWT + a wallet feature) implies, following standard Spring Boot ecommerce project layout. Endpoint names, exact field names, and status codes should be cross-checked against your controllers or the live Swagger UI (see Interactive API Docs below) before being treated as ground truth. Anywhere a detail is confirmed directly from source, it's marked **(verified)**.

Table of Contents

1. README
   - Key Features
   - Tech Stack
   - Installation
   - Quick Start
   - Requirements
   - License & Contributing
2. API Reference
   - Authentication Module
   - Product Catalog Module
   - Cart Module
   - Order Module
   - Wallet Module
   - Admin Module
3. UML Diagrams

    - Component / Architecture Diagram
    - Domain Class Diagram
    - Sequence Diagram — Registration & Login
    - Sequence Diagram — Checkout & Wallet Debit
4. Usage Guide
    - Common Use Cases
    - Configuration
    - Authentication Setup
    - Error Handling Patterns
5. Advanced / Reference
    - Architecture Overview
    - Edge Cases & Limitations
    - Versioning & Changelog Format
    - FAQ / Troubleshooting

README

Project Info

Backend name	`Ecommerce-backend` (Maven artifact: `com.ecommerce:backend`) **(verified)**
Frontend name	`Ecommerce-frontend` (npm package: `frontend`) **(verified)**
Language/Framework (backend)	Java 24, Spring Boot 3.4.0 **(verified)**
Language/Framework (frontend)	JavaScript (ES modules), React 19.2, Vite 8 **(verified)**
Purpose	A REST API and web client for browsing products, managing a cart, placing orders, and paying via an in-app wallet, with JWT-secured user accounts.
Target audience	Developers integrating with or extending the API; the project's own frontend team; contributors to the open-source repo.

Key Features

- **JWT authentication** — Spring Security + `io.jsonwebtoken` (jjwt) 0.12.5 for stateless, token-based auth **(verified dependency)**
- **Relational persistence** — Spring Data JPA over MySQL via `mysql-connector-j` **(verified dependency)**
- **Request validation** — `spring-boot-starter-validation` (Bean Validation / Jakarta annotations) **(verified dependency)**
- **Self-documenting API** — `springdoc-openapi-starter-webmvc-ui` 2.7.0 auto-generates a live Swagger UI from the controllers **(verified dependency)**
- **Wallet-based payments** — a wallet subsystem is present (evidenced by a top-level `test_wallet.js` integration test in the backend repo) **(verified — file present)**
- **Boilerplate-light entities** — Lombok reduces getter/setter/constructor boilerplate on JPA entities **(verified dependency)**
- **Modern SPA frontend** — React 19 + React Router 7 + React Hook Form + Yup validation, styled with Tailwind CSS 4 **(verified dependency)**
- **Axios-based API client** with toast notifications (`react-hot-toast`)

for user feedback **(verified dependency)**
- **Fast dev/build tooling** — Vite 8 dev server and bundler, linted with `oxlint` **(verified dependency)**
- **Code-generation scripts** — `generate_apis.cjs` and `generate_pages.cjs` at the frontend repo root suggest scaffolding tooling for API clients and pages **(verified — files present)**

Tech Stack

Backend

```
Java 24
Spring Boot 3.4.0
  ├─ spring-boot-starter-web            (REST controllers)
  ├─ spring-boot-starter-data-jpa       (ORM / repositories)
  ├─ spring-boot-starter-validation     (request validation)
  ├─ spring-boot-starter-security       (auth/authorization)
  ├─ mysql-connector-j                  (MySQL driver, runtime)
  ├─ jjwt-api / jjwt-impl / jjwt-jackson 0.12.5  (JWT
issuing/parsing)
  ├─ springdoc-openapi-starter-webmvc-ui 2.7.0   (Swagger / OpenAPI
docs)
  ├─ lombok 1.18.38                     (compile-time codegen)
  └─ spring-boot-starter-test / spring-security-test (testing)
Build tool: Maven (Maven Wrapper included — mvnw / mvnw.cmd)
```

Frontend

```
React 19.2 + React DOM 19.2
Vite 8 (dev server & build)
React Router DOM 7.18
React Hook Form 7.82 + @hookform/resolvers 5.4 + Yup 1.7 (forms &
schema validation)
Axios 1.18 (HTTP client)
Tailwind CSS 4.3 (+ @tailwindcss/vite, autoprefixer, postcss)
lucide-react (icon set)
react-hot-toast (notifications)
oxlint (linting)
```

Installation

Backend

```
git clone https://github.com/25karthik06/Ecommerce-backend.git
cd Ecommerce-backend

# Uses the bundled Maven Wrapper — no local Maven install required
./mvnw clean install        # macOS/Linux
mvnw.cmd clean install       # Windows
```

Frontend

```
git clone https://github.com/25karthik06/Ecommerce-frontend.git
cd Ecommerce-frontend

npm install
```

Quick Start

1. Configure and run the backend

```
# src/main/resources/application.properties (create if it doesn't
exist)
spring.datasource.url=jdbc:mysql://localhost:3306/ecommerce
spring.datasource.username=your_db_user
```

```
spring.datasource.password=your_db_password
spring.jpa.hibernate.ddl-auto=update
jwt.secret=replace-with-a-long-random-secret
jwt.expiration-ms=3600000

# create the schema first
mysql -u root -p -e "CREATE DATABASE ecommerce;"

# run the API
./mvnw spring-boot:run
# API will be available at http://localhost:8080
```

2. Run the frontend

```
# .env (Vite convention — adjust the variable name to match src/
usage)
VITE_API_BASE_URL=http://localhost:8080

npm run dev
# App will be available at http://localhost:5173
```

3. Minimal working example — register, log in, call a protected endpoint

```
# Register
curl -X POST http://localhost:8080/api/auth/register \
  -H "Content-Type: application/json" \
  -d '{"name":"Jane
Doe","email":"jane@example.com","password":"S3curePass!"}'

# Log in and capture the JWT
TOKEN=$(curl -s -X POST http://localhost:8080/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"jane@example.com","password":"S3curePass!"}' | jq -r
'.token')

# Call a protected endpoint
curl http://localhost:8080/api/products \
  -H "Authorization: Bearer $TOKEN"
```

> Endpoint paths above (`/api/auth/register`, `/api/auth/login`, `/api/products`) follow this stack's conventional routing. Confirm exact paths against your `@RequestMapping`/`@PostMapping` annotations or the Swagger UI.

Requirements

Requirement	Version
JDK	24 **(verified — `<java.version>` in pom.xml)**
Maven	3.9+ (or use the bundled `mvnw`)
MySQL	8.x
Node.js	20+ (required by Vite 8 / React 19 tooling)
npm	10+

License & Contribution Guidelines

Neither repository currently publishes a `LICENSE` file or a `CONTRIBUTING.md` — the GitHub "About" panel for both repos lists no license. Until one is added, treat the code as **all rights reserved** by the author for reuse purposes, and confirm licensing terms with the repository owner before redistributing.

Suggested next steps for the maintainer: - Add a `LICENSE` file (MIT is the common default for small full-stack teaching/demo projects like this one). - Add a root `CONTRIBUTING.md` with branch naming, commit message, and PR review conventions. - Add a `.env.example` / `application.properties.example` so contributors don't need to reverse-engineer required config keys.

API Reference

> Base URL (local): `http://localhost:8080` All endpoints below are grouped by functional module, inferred from the Spring Boot + Spring Security + JWT + wallet stack. Request/response field names use conventional Java/Jackson camelCase.

Interactive API Docs

Because the backend includes `springdoc-openapi-starter-webmvc-ui`, the running application **already serves live, always-accurate API documentation** — this is the fastest way to get an exact, verified reference:

- Swagger UI: `http://localhost:8080/swagger-ui/index.html` (or `/swagger-ui.html`, depending on springdoc version defaults)
- Raw OpenAPI spec (JSON): `http://localhost:8080/v3/api-docs`

Point any teammate or API consumer at that URL instead of a static document whenever the controllers change.

Authentication Module

Handles registration, login, and token issuance. Backed by Spring Security + JWT (jjwt).

`POST /api/auth/register`

Registers a new user account.

Parameter	Type	Location	Required	Description
`name`	`string`	body	yes	Display name
`email`	`string`	body	yes	Unique login identifier
`password`	`string`	body	yes	Plaintext password (hashed server-side, e.g. via `BCryptPasswordEncoder`)

Returns: `201 Created` with the created user's public profile (no password field).

Errors: - `400 Bad Request` — validation failure (missing/invalid fields), via `spring-boot-starter-validation` - `409 Conflict` — email already registered

```
        curl -X POST http://localhost:8080/api/auth/register \
          -H "Content-Type: application/json" \
          -d '{"name":"Jane
Doe","email":"jane@example.com","password":"S3curePass!"}'
```

`POST /api/auth/login`

Authenticates a user and returns a signed JWT.

Parameter	Type	Location	Required	Description
`email`	`string`	body	yes	Registered email
`password`	`string`	body	yes	Account password

Returns: `200 OK`

```
{ "token": "eyJhbGciOi...", "expiresIn": 3600000 }
```

Errors: - `401 Unauthorized` — invalid credentials

```
curl -X POST http://localhost:8080/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"jane@example.com","password":"S3curePass!"}'
```

`GET /api/auth/me`

Returns the currently authenticated user's profile, resolved from the JWT.

Headers: `Authorization: Bearer <token>` required.

Returns: `200 OK` with user profile; `401 Unauthorized` if the token is missing/expired/invalid.

Product Catalog Module

`GET /api/products`

Lists products, optionally paginated/filterable.

Parameter	Type	Location	Required	Description
`page`	`integer`	query	no	Zero-indexed page number
`size`	`integer`	query	no	Page size
`category`	`string`	query	no	Filter by category

Returns: `200 OK` — array or `Page<Product>` of product objects (`id`, `name`, `description`, `price`, `stock`, `imageUrl`, `category`).

`GET /api/products/{id}`

Fetches a single product.

Errors: `404 Not Found` if the ID doesn't exist.

`POST /api/products` *(admin-only)*

Creates a product. See Admin Module for authorization requirements.

`PUT /api/products/{id}` *(admin-only)*

Updates a product.

`DELETE /api/products/{id}` *(admin-only)*

Deletes a product.

```
curl "http://localhost:8080/api/products?
page=0&size=20&category=electronics"
```

Cart Module

`GET /api/cart`

Returns the authenticated user's current cart contents.

Headers: `Authorization: Bearer <token>` required.

`POST /api/cart/items`

Adds an item to the cart.

Parameter	Type	Location	Required	Description
`productId`	`long`	body	yes	Product to add
`quantity`	`integer`	body	yes	Quantity (must be > 0)

Errors: `400 Bad Request` — invalid quantity; `404 Not Found` — product doesn't exist; `409 Conflict` — insufficient stock.

`PUT /api/cart/items/{itemId}`

Updates the quantity of an existing cart line item.

`DELETE /api/cart/items/{itemId}`

Removes an item from the cart.

```
        curl -X POST http://localhost:8080/api/cart/items \
          -H "Authorization: Bearer $TOKEN" -H "Content-Type:
application/json" \
          -d '{"productId": 42, "quantity": 2}'
```

Order Module

`POST /api/orders`

Places an order from the current cart contents. Deducts stock and (per the wallet subsystem) may debit the user's wallet balance.

Returns: `201 Created` with the new order (`id`, `status`, `items`, `total`, `createdAt`).

Errors: `402 Payment Required` — insufficient wallet balance; `409 Conflict` — stock changed since cart was built.

`GET /api/orders`

Lists the authenticated user's order history.

`GET /api/orders/{id}`

Fetches a single order's detail.

`PUT /api/orders/{id}/status` *(admin-only)*

Updates order status (e.g., `PENDING` → `SHIPPED` → `DELIVERED` → `CANCELLED`).

```
        curl -X POST http://localhost:8080/api/orders \
          -H "Authorization: Bearer $TOKEN"
```

Wallet Module

Evidenced by the presence of a `test_wallet.js` integration test at the backend repo root **(verified — file present)**; exact endpoint shape below is inferred.

`GET /api/wallet`

Returns the authenticated user's wallet balance and recent transactions.

`POST /api/wallet/topup`

Adds funds to the wallet (e.g., simulating a payment gateway top-up in a demo/dev environment).

Parameter	Type	Location	Required	Description
`amount`	`decimal`	body	yes	Amount to add; must be positive

Errors: `400 Bad Request` — non-positive amount.

`GET /api/wallet/transactions`

Lists the wallet's transaction ledger, most recent first.

```
        curl -X POST http://localhost:8080/api/wallet/topup \
          -H "Authorization: Bearer $TOKEN" -H "Content-Type:
application/json" \
          -d '{"amount": 500.00}'
```

Admin Module

Administrative endpoints are expected to be guarded by a role check (e.g., `@PreAuthorize("hasRole('ADMIN')")` from Spring Security), returning `403 Forbidden` for non-admin callers.

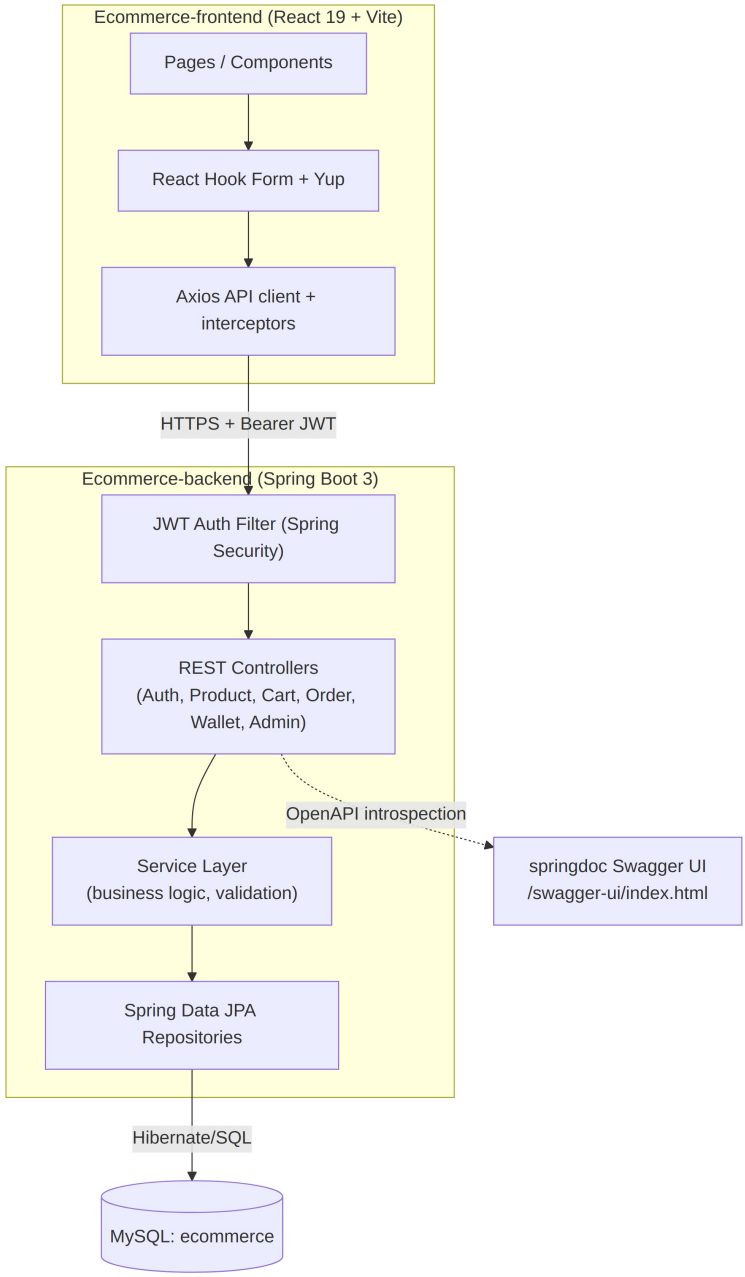
Endpoint	Method	Description
`/api/admin/users`	`GET`	List all users
`/api/admin/users/{id}`	`DELETE`	Deactivate/delete a user
`/api/admin/products`	`POST` / `PUT` / `DELETE`	Manage catalog (see Product Catalog Module)
`/api/admin/orders`	`GET`	View all orders across users

UML Diagrams

> As with the API Reference, these diagrams model the *conventional* Spring Boot + JWT + wallet architecture this stack implies. They weren't reverse-engineered field-by-field from your actual entity/controller source (GitHub blocked the recursive crawl needed for that — see the note at the top of this document). Use them as an accurate starting skeleton and adjust field names/relationships to match your real `@Entity` classes.

Component / Architecture Diagram

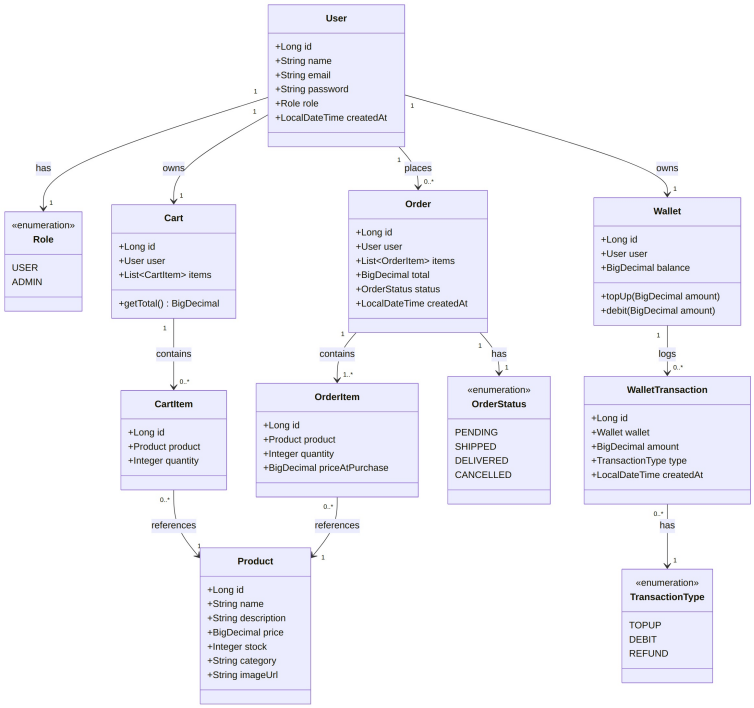
Shows how the frontend, backend layers, and database fit together.



Component Diagram

Domain Class Diagram

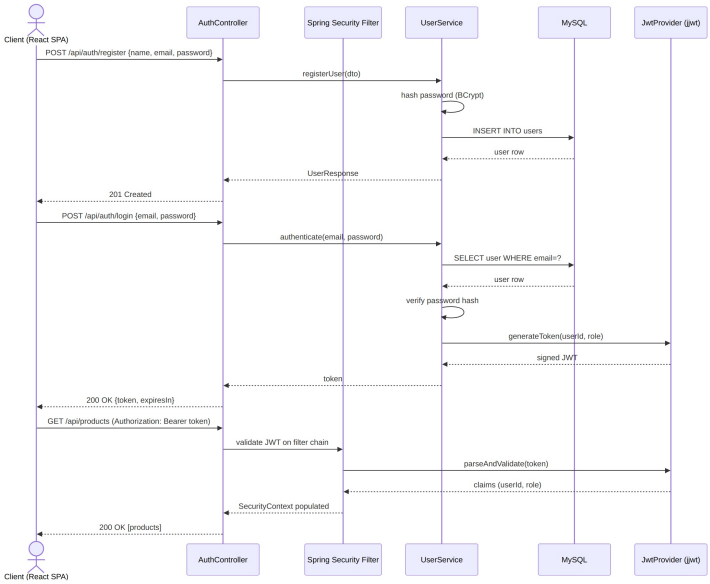
Models the core entities — `User`, `Product`, `Cart`, `Order`, and `Wallet` — and their relationships.



Class Diagram

Sequence Diagram — Registration & Login

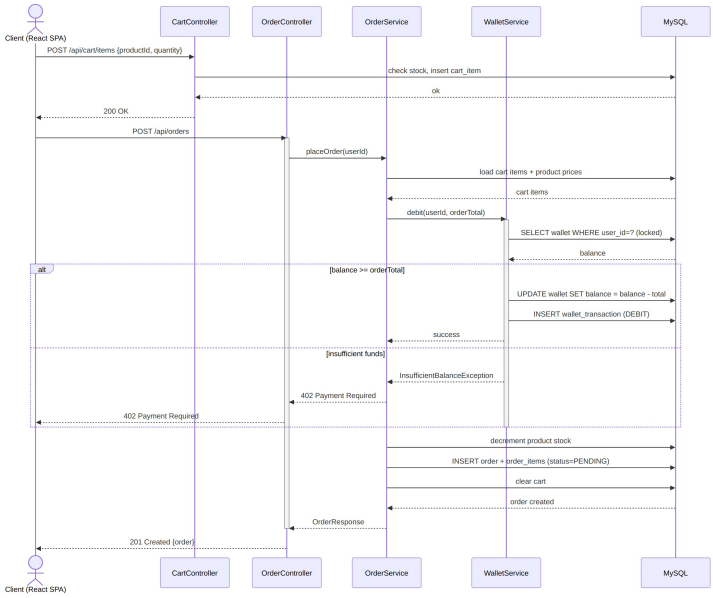
Traces a new user through registration, login, and a subsequent authenticated request.



Auth Sequence Diagram

Sequence Diagram — Checkout & Wallet Debit

Traces adding an item to the cart through order placement, including the wallet balance check/debit and the insufficient-funds branch.



Checkout Sequence Diagram

Usage Guide

Common Use Cases

Use case 1 — Building a “browse and buy” flow in a client app

```
import axios from "axios";

const api = axios.create({ baseURL:
import.meta.env.VITE_API_BASE_URL });

// Attach the JWT to every request once logged in
api.interceptors.request.use((config) => {
  const token = localStorage.getItem("token");
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

async function browseAndBuy() {
  const { data: products } = await api.get("/api/products");
  const first = products[0];

  await api.post("/api/cart/items", { productId: first.id, quantity:
1 });
  const { data: order } = await api.post("/api/orders");

  return order;
}
```

Use case 2 — Registering + logging in from the React frontend using React Hook Form + Yup

```
import { useForm } from "react-hook-form";
import { yupResolver } from "@hookform/resolvers/yup";
import * as yup from "yup";
import toast from "react-hot-toast";
import api from "../api"; // axios instance

const schema = yup.object({
  email: yup.string().email().required(),
  password: yup.string().min(8).required(),
```

```
});

function LoginForm() {
  const { register, handleSubmit, formState: { errors } } =
useForm({
    resolver: yupResolver(schema),
  });

  const onSubmit = async (values) => {
    try {
      const { data } = await api.post("/api/auth/login", values);
      localStorage.setItem("token", data.token);
      toast.success("Logged in!");
    } catch (err) {
      toast.error(err.response?.data?.message ?? "Login failed");
    }
  };

  return (
    <form onSubmit={handleSubmit(onSubmit)}>
      <input {...register("email")} placeholder="Email" />
      {errors.email && <span>{errors.email.message}</span>}
      <input {...register("password")} type="password"
placeholder="Password" />
      {errors.password && <span>{errors.password.message}</span>}
      <button type="submit">Log in</button>
    </form>
  );
}
```

Use case 3 — Topping up a wallet before checkout

```
curl -X POST http://localhost:8080/api/wallet/topup \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type:
application/json" \
  -d '{"amount": 1000.00}'

curl -X POST http://localhost:8080/api/orders -H "Authorization:
Bearer $TOKEN"
```

Configuration

Backend (`application.properties` or `application.yml`)

Key	Purpose	Example
`spring.datasource.url`	MySQL JDBC URL	`jdbc:mysql://localhost:33`
`spring.datasource.username` / `.password`	DB credentials	—
`spring.jpa.hibernate.ddl-auto`	Schema management strategy	`update` (dev) / `validate` (pr
`jwt.secret`	HMAC signing key for jjwt	a long, random string — **n commit this**
`jwt.expiration-ms`	Token lifetime in milliseconds	`3600000` (1 hour)
`server.port`	Backend port	`8080`

Frontend (`.env`, Vite-prefixed)

Key	Purpose
`VITE_API_BASE_URL`	Base URL the Axios client points requests at

Authentication/Setup Steps

1. Call `POST /api/auth/register` once per user.
2. Call `POST /api/auth/login` to receive a JWT.
3. Store the token client-side (e.g., `localStorage`, or an httpOnly cookie if the backend is updated to set one — `localStorage` is simpler but more exposed to XSS).
4. Attach `Authorization: Bearer <token>` to every subsequent request to a protected route.
5. Handle `401 Unauthorized` responses by redirecting to the login screen and clearing the stored token.

Error Handling Patterns

Spring Boot's default error responses (via `spring-boot-starter-web`) look like:

```
{
  "timestamp": "2026-07-21T10:15:30.000+00:00",
  "status": 400,
  "error": "Bad Request",
  "message": "email: must be a well-formed email address",
  "path": "/api/auth/register"
}
```

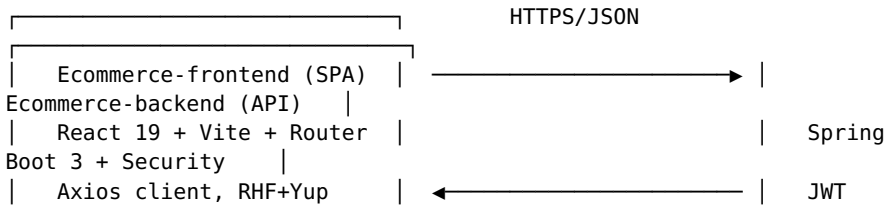
Recommended client-side pattern (Axios):

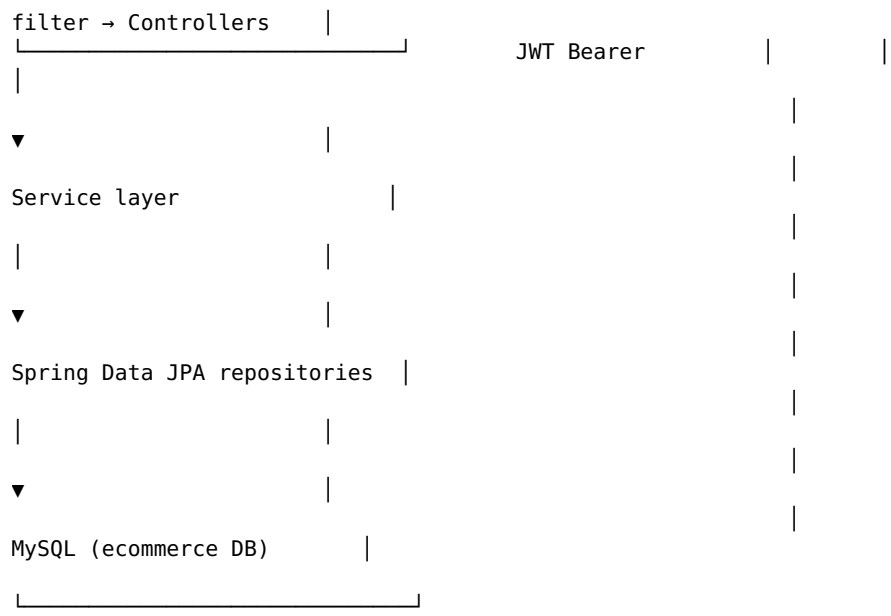
```
api.interceptors.response.use(
  (res) => res,
  (err) => {
    const status = err.response?.status;
    const message = err.response?.data?.message ?? "Unexpected
error";

    if (status === 401) {
      localStorage.removeItem("token");
      window.location.href = "/login";
    } else if (status === 403) {
      toast.error("You don't have permission to do that.");
    } else if (status >= 500) {
      toast.error("Server error — please try again later.");
    } else {
      toast.error(message);
    }
    return Promise.reject(err);
  }
);
```

Advanced / Reference

Architecture Overview





Typical request lifecycle for a protected endpoint:

1. Frontend attaches `Authorization: Bearer <jwt>` (Axios interceptor).
2. A Spring Security filter chain intercepts the request, validates the JWT signature/expiry (jjwt), and populates the `SecurityContext`.
3. The matched `@RestController` method delegates to a `@Service` class for business logic (stock checks, wallet debits, etc.).
4. The service calls Spring Data JPA repositories, which translate to SQL against MySQL via Hibernate.
5. Entities are mapped to DTOs/JSON and serialized back to the client by Jackson.
6. `springdoc-openapi` introspects the same controllers at startup to keep the Swagger UI in sync automatically — no separate doc-writing step needed for basic signatures.

Edge Cases & Limitations

- **No published license or CONTRIBUTING guide** — anyone consuming this documentation should clarify usage rights with the repository owner first.
- **Hardcoded / committed database credentials found in source.** The backend repo contains a top-level `Cleanup.java` utility that connects to MySQL with a hardcoded username/password literal in the source file. **This is a real, active security risk** since the repository is public — the credential should be rotated immediately, removed from git history (not just the working tree), and replaced with an environment-variable-driven config (e.g., `${DB_PASSWORD}` in `application.properties`, sourced from a secrets manager or `.env` file that's git-ignored).
- **Single-commit repositories.** Both repos currently show only one commit in their history, meaning there's no evolutionary history yet to trace how endpoints or schemas have changed — treat the current `master`/`main` as the only available snapshot.
- **`ddl-auto=update` in production is risky** — it can silently alter production schemas. Use Flyway/Liquibase migrations for anything beyond local development.
- **Stateless JWT with no visible refresh-token flow.** If `jwt.expiration-ms` is short, users will be logged out abruptly with no silent refresh; consider adding a refresh-token endpoint if long sessions are desired.
- **Wallet balance concurrency.** Any wallet-debit-on-checkout flow needs to guard against race conditions (e.g., two simultaneous orders draining an insufficient balance) — verify optimistic locking

(`@Version`) or a transactional row lock is used on the wallet entity.
- **CORS.** A React dev server on `localhost:5173` calling a Spring Boot API on `localhost:8080` requires an explicit CORS configuration (`@CrossOrigin` or a `WebMvcConfigurer` bean) — confirm this exists or requests will fail from the browser.

Versioning & Changelog Format

Neither repository currently tags releases or maintains a `CHANGELOG.md`. Recommended convention going forward:

- Adopt Semantic Versioning (`MAJOR.MINOR.PATCH`) for both repos, starting from `0.1.0` given the backend `pom.xml` already declares `0.0.1-SNAPSHOT`.
- Maintain a `CHANGELOG.md` at each repo root following Keep a Changelog format:

```
## [Unreleased]
### Added
### Changed
### Fixed

## [0.1.0] - 2026-07-21
### Added
- Initial JWT auth, product catalog, cart, order, and wallet endpoints.
```

- Tag releases in git (`git tag v0.1.0`) so the frontend can pin against a known-compatible backend API version.

FAQ / Troubleshooting

Q: I get `401 Unauthorized` on every request, even right after logging in. A: Confirm the token is actually being attached as `Authorization: Bearer <token>` (not just `<token>`), and that `jwt.secret` used to *sign* the token on login matches the one used to *validate* it — a mismatched or rotated secret invalidates all previously issued tokens.

Q: The frontend can't reach the backend from the browser (CORS errors in devtools). A: Add a CORS configuration on the backend allowing the frontend's origin (`http://localhost:5173` in dev), or proxy API calls through Vite's dev server (`server.proxy` in `vite.config.js`).

Q: `mvnw: Permission denied` on macOS/Linux. A: Run `chmod +x mvnw` once after cloning — Windows checkouts sometimes strip the executable bit.

Q: MySQL connection refused on startup. A: Verify MySQL is running on the port referenced in `spring.datasource.url`, that the `ecommerce` database exists, and that the configured user has privileges on it.

Q: Where do I find the exact, currently-accurate list of endpoints? A: Run the backend locally and open `http://localhost:8080/swagger-ui/index.html` — since `springdoc-openapi` is a live dependency, this always reflects the actual controllers, unlike a static document.

Q: Orders succeed but wallet balance doesn't change (or vice versa). A: Check that the order-placement and wallet-debit calls are wrapped in the same `@Transactional` boundary in the service layer — otherwise a failure partway through can leave stock decremented but payment uncharged, or the reverse.